SALESBENCH: A Long-Horizon, Agent-to-Agent Benchmark for Tool-Use Agent Behavior

Anonymous authors
Paper under double-blind review

Abstract

Most agent benchmarks ask *what* an agent achieves against a static rubric or fixed verifier. For deployed agents, an equally important question is *how* they behave: how they allocate a budget, where they break, whether they remain compliant under optimization pressure, and whether a learned policy survives a counterparty it did not train against. We introduce SALESBENCH, a long-horizon, agent-to-agent benchmark for measuring those behaviors. A seller agent operates inside a deterministic insurance-sales simulator with ∼100 procedurally generated leads and a fixed time budget. It must maximize converted monthly recurring revenue by searching a customer database, quoting plans, placing calls, and closing deals, while a separate buyer language model, instantiated with one of ten personality archetypes, decides whether to accept, reject, or hang up. Because the world is a deterministic state machine, variance comes from the seller and buyer models; seller mistakes are separated from buyer-model failures; and four buyer-prompt variants turn counterparty overfitting into a quantitative test. We use SALESBENCH as both a training signal and an evaluation suite, training a 2B-parameter model with online reinforcement learning and analyzing its behavior. The trained policy learns in two phases, tool-interface mastery then selling skill, and at full scale is productive but brittle: it converts ∼18 leads using only ∼11% of the time budget before terminating on the invalid-action ceiling rather than on time. Compliance holds under revenue optimization (do-not-call violations stay near zero), and the policy transfers to three unseen buyer decision styles (8–20% per-lead conversion), degrading less than the base model as the pipeline doubles. Finally, reward shaping is fragile under RL: auxiliary terms become floor strategies once the model can collect them without selling. We release the environment, curriculum, behavioral instrumentation, and harness.

1 Introduction

Language-model agents are deployed on tasks where success is not a single correct answer but the cumulative outcome of many decisions made under a budget. A coding agent navigates a repository across dozens of tool calls; a research agent reads, synthesizes, and revises; a customer-facing agent negotiates across a multi-turn conversation. Yet many agent benchmarks retain the shape of question answering: a prompt, a trajectory, and a verifier that scores the trajectory against a fixed target (Liu et al., 2024b; Mialon et al., 2024). This format is convenient because it is deterministic and gradeable, but it mostly measures *what* an agent achieves. It says less about *how* the result was produced, which is the process question that determines whether an agent is robust, compliant, and useful under deployment pressure.

We build on a line that evaluates agents through a behavioral lens (Park et al., 2023; Zhou et al., 2024b; Pan et al., 2023): evaluating the *behavior* of an agent requires two ingredients that single-trajectory benchmarks often omit. The first is **horizon under a hard budget**. A real agent does not get an arbitrary number of actions; it has a finite amount of time, money, context, or tolerance for error, and good behavior means *allocating* that budget, deciding which leads to pursue, when to stop talking and act, and when to cut losses.

The second is the **agent-to-agent** structure. The environment is not a passive grader but another adaptive agent with its own goals, information, and personality, whose responses the protagonist cannot dictate and must instead influence. Sales, negotiation, tutoring, and customer support all have this structure, and all are domains where current models are being deployed with little principled measurement of how they behave under pressure.

Existing benchmarks tend to supply one ingredient but not both: long-horizon business simulators are stateful and budgeted but pair the agent with scripted customers, while tool-agent-user benchmarks supply an adaptive counterparty in shorter episodes. SALESBENCH sits at the intersection: long-horizon, stateful, budgeted, driven by an adaptive language-model counterparty, and graded by a verifiable outcome read directly from environment state, namely whether a deal closed and at what premium.

We introduce SALESBENCH, a benchmark that captures both ingredients in a single reproducible task, simulated insurance sales (§3). A seller agent works a pool of ∼100 prospective buyers ("leads") under a hard budget of, e.g., 50 simulated working hours, spending simulated minutes on each action (searching a customer-relationship-management (CRM) system, pulling a quote, starting a call, proposing an offer) to maximize converted monthly recurring revenue (MRR), the sum of premiums on deals it closes. Closing requires a separate buyer language model, instantiated per-lead with one of ten personality archetypes (Analytical, Skeptic, Budget Hawk, Protector, and so on) and a "temperature" from cold to hot, to accept the offer; the buyer decides, in character, whether to accept, reject, or hang up.

This design yields three properties a behavioral benchmark should have. **Reproducible world, stochastic agents**: the simulator (§3) is a deterministic state machine with zero model calls, so for a fixed seed the only variance is the seller and buyer models, which makes k-rollout behavioral comparison meaningful. **Failure attribution**: SALESBENCH separates a protagonist's bad decisions from counterparty-model failures (§3.6), penalizing seller errors while resolving buyer-model failures to a benign rejection. **A built-in generalization axis**: four buyer-prompt variants share identical information but differ in decision style, so an agent trained against one can be tested against the other three.

To show the benchmark is a trainable signal rather than only a static evaluation set, we run a four-stage difficulty curriculum (2, 4, 8, then 20 leads) that trains a 2B-parameter model with online reinforcement learning against the DEFAULT buyer, then evaluate it at full scale (50 and 100 leads) against all four buyers (§4). Rather than reporting a single capability number, we use the run to surface the behaviors the benchmark is meant to measure:

- **Two-phase learning.** The agent first masters the tool *interface* (schema errors fall sharply at small scale), and only then, much more slowly, learns the *skill* of closing: fitting coverage to budget and triaging which leads to pursue (§4.3).
- **A brittleness signature.** At 100 leads the trained agent uses only ∼11% of its time budget: it converts ∼18 leads and then trips a mistake ceiling, because interface discipline learned at small scale does not fully transfer to the long horizon (§4.4).
- **Compliance under optimization.** Do-not-call violations stay near zero even as the agent is optimized hard for revenue (§4.4).
- **Counterparty-style generalization.** Trained only against DEFAULT, the agent sustains 8–20% per-lead conversion against all four buyer styles, including three it never saw. This suggests a transferable policy, not counterparty overfitting (§4.5).
- **Reward-shaping fragility.** Almost any auxiliary reward term becomes a floor trap once the model can collect it without selling, so SALESBENCH doubles as a testbed for reward-hacking behavior (§5).

Our contributions are: (1) SALESBENCH, a reproducible long-horizon agent-to-agent benchmark for *measuring tool-use agent behavior*, with explicit seller/buyer failure attribution and a buyer-style generalization axis; (2) a deterministic simulator, ten-archetype buyer model, revenue-first reward, difficulty curriculum, and zero-weight behavioral instruments for termination attribution, pipeline triage, and per-counterparty conversion fingerprints; (3) a behavioral characterization of a small RL-trained agent: two-phase learning, an error-ceiling brittleness signature, compliance under optimization, and counterparty-style transfer.

2 Related work

Agentic and tool-use benchmarks. A growing body of work evaluates language models as agents that call tools over multiple turns: AgentBench (Liu et al., 2024b) and GAIA (Mialon et al., 2024) aggregate diverse tasks; ToolLLM (Qin et al., 2024) and Gorilla (Patil et al., 2024) target API invocation; WebArena, SWE-bench, and TheAgentCompany (Zhou et al., 2024a; Jimenez et al., 2024; Xu et al., 2024) study realistic web, software, and professional environments. These are mostly graded against a static target (a gold trajectory, a passing test suite, or a reachable goal), and even analytical multi-turn boards such as AgentBoard (Ma et al., 2024) score progress against fixed tasks. SALESBENCH instead uses an adaptive counterparty, a continuous resource-allocation outcome, and rollout instrumentation for studying *how* the policy behaves.

Agent behavior and social simulation. A complementary line studies agents through a behavioral lens: Generative Agents (Park et al., 2023) and Concordia (Vezhnevets et al., 2023) simulate language-model personas; SOTOPIA (Zhou et al., 2024b) evaluates social intelligence in agent-to-agent episodes; MACHIAVELLI (Pan et al., 2023) measures the reward-versus-ethics trade-off; and model-written evaluations (Perez et al., 2022) and sycophancy (Sharma et al., 2024) surface emergent behaviors induced by optimization. SALESBENCH adds a long-horizon, economically grounded setting where budget allocation, compliance under pressure, counterparty influence, and brittleness are read directly from a deterministic environment rather than a judge.

Agent-to-agent and tool-agent-user simulation. The closest line of work simulates a second agent as part of the environment: τ-bench and τ^2-bench (Yao et al., 2024; Barres et al., 2025) introduce a user-simulator language model the agent must serve under domain policies, with the three-party structure (agent, simulated user, environment) we adopt. SALESBENCH shares this pattern but targets a different regime: the counterparty can reject or hang up, the objective is competitive revenue maximization rather than policy-compliant completion, and the horizon spans hundreds of tool calls across $\sim$100 counterparties under one budget. Negotiation and persuasion (Lewis et al., 2017; Wang et al., 2019; Meta Fundamental AI Research Diplomacy Team (FAIR) et al., 2022) study influence inside a single conversation; SALESBENCH embeds many such conversations in a portfolio-level budgeting problem.

Long-horizon, budgeted agents. Vending-Bench (Backlund & Petersson, 2025) runs a simulated business over long horizons with persistent state and a budget, closest to ours in operational shape, but its customers are scripted, whereas τ-bench's counterparty is adaptive but its episodes are short. SALESBENCH occupies the intersection: long-horizon, stateful, adaptive-counterparty, resource-constrained, and graded by a verifiable state-derived outcome with no judge.

Reinforcement learning and behavioral intervention. Reinforcement learning from verifiable rewards has driven progress in reasoning and instruction following (Shao et al., 2024; Lambert et al., 2024); we extend it to a long-horizon, agent-to-agent task with GRPO (Shao et al., 2024) and a difficulty curriculum (Bengio et al., 2009). We treat RL post-training as a *behavioral intervention*: it installs an operational selling policy but also exposes reward-shaping fragility, a concrete instance of reward misspecification and reward hacking (Pan et al., 2022; Skalse et al., 2022; Krakovna et al., 2020; Pan et al., 2024). The environment is built on VERIFIERS (Brown, 2025), whose stateful tool-environment abstraction supports the per-rollout simulator state and hidden tool arguments we require.

3 The SALESBENCH environment

3.1 Task and objective

An episode places a seller agent in a sales pipeline defined by a seed; it receives a brief and a tool set, then acts turn by turn until termination. The world is a population of N

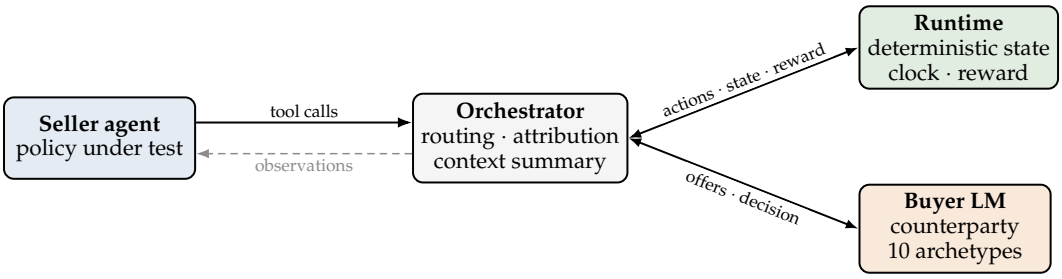

Figure 1: Three-party orchestration (the released harness). The seller agent talks only to the orchestrator, exchanging tool calls and observations. The orchestrator applies validated actions to a deterministic runtime and routes proposed offers to a buyer language model, then injects the buyer's decision and speech back into the conversation. Seller errors are penalized and increment the invalid-action counter; buyer-model failures are isolated and resolve to a benign rejection (§3.6).

leads (default N=100), a catalog of four insurance plan types, a wall clock with budget $B = H \times 60$ (default H=50 hours, B=3000 minutes), a per-action time-cost schedule, and a hard cap on accumulated invalid actions (default 12). Revenue accrues only when the buyer accepts an offer, adding the accepted premium p_i to MRR. A lead's monthly budget b_i is not a hard cap: whether an over-budget offer is accepted depends on archetype and temperature, so pricing to budget is a learned skill. The episode ends when *any* budget is exhausted: time, pipeline (all leads converted, on the do-not-call list, or out of attempts), or the mistake budget (invalid-action cap). Which budget binds is itself a behavioral outcome we measure (§4.4).

3.2 Three-party orchestration

SALESBENCH follows the agent-counterparty-environment decomposition of τ^2-bench (Barres et al., 2025), separated into three components with disjoint responsibilities (Figure 1). The **runtime** is a pure Python state machine that owns all episode state, leads, clock, active call, callbacks, statistics, and termination, and contains zero model calls. Determinism is enforced at construction (a single seeded RNG produces the lead population; prices are closed-form), and we verify reproducibility (§A.10). The **buyer policy** is a separate language model: on each proposed offer it is prompted with the lead's full persona and the call transcript and returns a structured decision in {accept, reject, hang_up} with a natural-language reason; during conversation it replies in character. The **orchestrator** routes the seller's tool calls into the runtime, injects the buyer's decision and speech back into the conversation, manages context length, enforces failure attribution, and instantiates one buyer policy per episode. The buyer is injected into the offer tool via hidden arguments, so the seller's tool schema never exposes the runtime or the buyer handle.

3.3 Leads, archetypes, and pricing

Leads are sampled from three demographic segments with correlated income, household size, and risk. Each lead draws a temperature (cold/lukewarm/warm/hot; near-uniform) that shifts trust and call tolerance, and one of ten buyer archetypes (Analytical, Skeptic, Budget Hawk, Protector, Impulse Decider, ...; uniform) that sets which of seven sales-quality criteria (discovery, tailoring, objection handling, pressure calibration, value articulation, rapport, pacing) the buyer weights when evaluating an offer. A lead's monthly budget is a fraction of its income drawn from a wide range, giving an even spread of deal sizes (in a representative 100-lead episode, budgets span \$52–\$952/month, max achievable MRR \$32,610). The catalog prices four plan types with a deterministic premium modulated by age, risk class, term, and smoking status, posing a real coverage-to-budget fitting problem (§A.5).

3.4 Tools and the canonical workflow

The seller is given eleven tools across four namespaces, CRM, calling, products, and calendar (full list and per-action time costs in §A.5). A productive call follows the pattern: start a call, briefly discover the need, pull a quote that fits the budget, propose the offer, end the call. Two invariants make the task honest: exactly one call may be active at a time, and `propose_offer` is rejected as an invalid action unless the agent has already pulled a quote for that lead (an anti-hallucination rail against fabricated prices). Proposing an offer costs the most time (four minutes), so the budget forces triage: the agent cannot call every lead, and long conversations crowd out closes.

3.5 Reward

The reward is a revenue-first weighted composite computed at episode end:

$$R = 1.00 \cdot \frac{\text{MRR}}{\text{MRR}_{\text{max}}} + 0.10 \cdot \frac{\text{conversions}}{N} + 0.30 \cdot \text{budget\_util} + 0.02 \cdot \text{completion} \\ - 0.30 \cdot \text{dnc\_violations} - 0.05 \cdot (0.1 \cdot \text{invalid\_actions}),$$

where MRR_{max} is the sum of lead budgets, budget_util rewards capturing a high share of each converted lead's budget, and the penalties cover compliance and malformed actions (full definitions in §A.2). Revenue dominates by design; the shaping terms are deliberately small (an earlier, larger completion bonus induced a degenerate floor strategy under GRPO, §5). All six weights are configurable, and the environment also logs ∼35 zero-weight diagnostic metrics plus the behavioral instruments of §3.7, so the reward and its telemetry are experimental variables.

3.6 Failure attribution

The orchestrator separates two failure classes so that scores reflect the agent's decisions, not the buyer's serving reliability. Seller errors (no active call, proposing without a quote, unknown plan type, calling a do-not-call lead) raise a deterministic error and increment the invalid-action counter; buyer-model failures (timeouts, malformed JSON, transport errors) are caught and resolved to a benign rejection, never counted against the seller. The offer tool enforces this by ordering record-offer (seller-attributable) → call-buyer (isolated) → apply-decision, so a buyer outage cannot manufacture an invalid action.

3.7 Behavioral instrumentation

Because the central question is *how* the agent behaves, the environment ships zero-weight instruments (no reward impact) that decompose each episode at termination: (i) *termination attribution*, the fraction of episodes ending via pipeline exhaustion, time exhaustion, the invalid-action ceiling, or no controlled stop at all; (ii) *pipeline triage*, the fraction of the lead pool contacted, and the warm/hot share among contacted leads (the pool is ∼51% warm/hot, so a higher share indicates the agent triages toward readier leads); and (iii) *per-counterparty conversion fingerprints*, conversion rate broken out by each of the ten archetypes and four temperatures. These instruments turn coarse outcomes into behavioral profiles and are what let us, in §4.4, distinguish "the agent ran out of time" from "the agent broke."

3.8 Context management for long horizons

At full scale an episode spans hundreds of tool calls and outgrows the context window, where models also degrade on information buried in the middle of long contexts (Liu et al., 2024a). SALESBENCH compresses deterministically: when the prompt reaches a configurable fraction (default 80%) of the maximum sequence length, the orchestrator replaces older messages with a compact summary rendered from runtime state, keeping the system prompt, briefing, and most recent messages verbatim (§A.4). The summary uses no model call and fires a handful of times per episode, keeping the key-value cache stable for training. This compression is load-bearing at 100 leads (§4.4).

4 Experiments

We use SALESBENCH both as a training signal and as an evaluation suite, to demonstrate the full loop the benchmark is designed to support and to read off the agent's behavior.

4.1 Setup

Curriculum. We train a 2B-parameter base model (Qwen3.5-2B) with Group Relative Policy Optimization (GRPO) (Shao et al., 2024) over four difficulty stages of increasing scale: 2 leads / 1 hour, 4 / 2 hours, 8 / 4 hours, and 20 / 10 hours. Only the first stage trains from scratch (to acquire the search→call→quote→propose workflow); each subsequent stage warm-starts from the previous checkpoint. Training uses the DEFAULT buyer exclusively, 32 rollouts per example for group-relative advantages, online difficulty filtering, and sampling temperature 1.0; the buyer model is a small hosted model held fixed throughout (full hyperparameters in §A.7).

Evaluation. We evaluate at full scale, 50 and 100 leads, which is 2.5–5× larger than any training stage. Each evaluation cell rolls out 64 episodes from the held-out evaluation split (a 128-seed split disjoint from training), one rollout each, with deterministic context compression at threshold 0.85 and maximum sequence length 16,000 tokens. Because each episode uses a distinct seed, the 64 episodes per cell are independent draws; we report the standard error of the mean (SEM), the sample standard deviation across episodes divided by $\sqrt{64}$, as $\pm$ values in Table 1 and error bars in Figure 2. We run the trained agent and the untrained base against all four buyer variants (DEFAULT, SKEPTICAL, IMPULSIVE, ANALYTICAL); the latter three are never seen in training. We report composite reward, per-lead conversion rate (conv/lead), captured MRR fraction (mrr-cap), and the fraction of episodes that reach a controlled runtime stop (done).

4.2 Main results

Table 1 reports the evaluation matrix. The trained agent improves over the untrained base on every cell and every metric. Against the DEFAULT buyer it raises per-lead conversion from 1.3% to 26.8% at 50 leads and from 0.3% to 18.5% at 100 leads, and composite reward from negative (-0.04) to positive ($+0.19$ to $+0.32$). Expressed as a ratio, the mean conversion lift over the untrained base is 21× at 50 leads and 50× at 100 leads; we report these lifts because they are striking, but note the ratio is inflated by a near-zero baseline (the untrained model converts almost nothing at scale) and is sensitive to baseline noise, which is why we foreground the absolute conversion rates and the signed composite reward throughout (Figure 2).

4.3 Learning dynamics: interface, then skill

The training run reveals a two-phase structure. The untrained base model fails the *interface* before it can attempt the task: in a representative diagnostic of three episodes it issued 43 offer attempts with zero quote calls and mis-filled the argument schema. It placed "ACCEPT" into the offer's next-step field, as if the buyer's decision were something the seller could set, and produced a single accidental conversion (Table 1). The first thing the agent learns (within roughly the first 50 optimization steps at the 2-lead stage) is the tool *syntax*: schema errors vanish and the mean invalid-action count per episode falls from about 6 to about 0.5. The second, much slower thing it learns is the *skill*, closing deals, which requires honoring the quote-before-propose constraint, fitting coverage to budget, and triage: recognizing when an objection is about information rather than price, and when to abandon a lead rather than burn four-minute offers retrying it. The curriculum makes this skill transfer efficiently: only the first 2-lead stage does substantial work (about 200 steps to near its ceiling), and the subsequent 4-lead stage starts already at 95.7% of its own ceiling, with later stages following the same fast-transfer pattern. Representative rollouts from the base, a mid-training checkpoint, and the trained policy (App. A.8) make the two phases concrete: at step 50 of the 2-lead stage the agent drives the tool loop cleanly (a quote before every

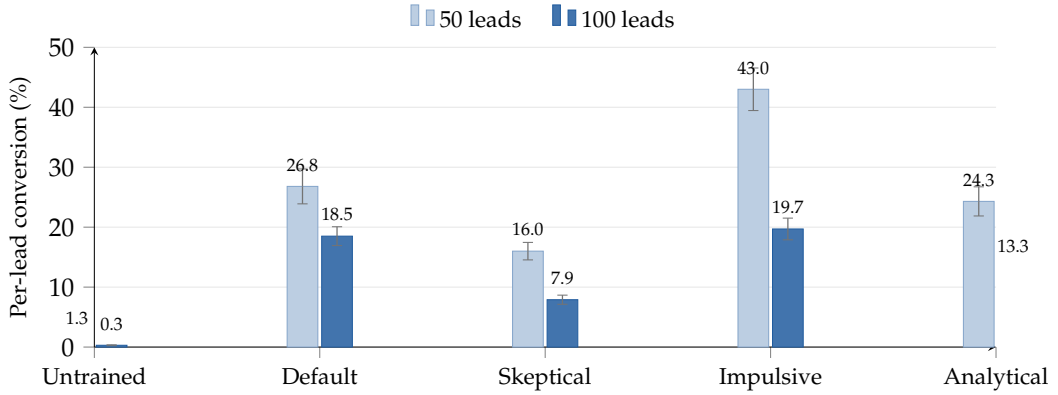

Figure 2: Model performance: per-lead conversion at 50 and 100 leads (64 episodes per cell; error bars are ± 1 SEM). The leftmost pair is the *untrained* base on the DEFAULT buyer; the remaining four are the *trained* agent evaluated against each buyer-prompt variant, where DEFAULT is the only buyer seen in training. Doubling the pipeline lowers every setting, but the trained policy stays well above the untrained base and holds 7.9–19.7% conversion across all four buyer styles. The ratio over the base rises from $\sim$21$\times$ at 50 leads to $\sim$50$\times$ at 100 because the trained burst is bounded while the near-zero baseline collapses faster (§4.4).

Scale	Agent / buyer	reward	conv/lead	mrr-cap	done
50 leads	untrained, DEFAULT	$-0.039_{\pm.003}$	$0.013_{\pm.004}$	0.002	0.469
	trained, DEFAULT	$0.316_{\pm.040}$	$0.268_{\pm.029}$	0.276	0.922
	trained, SKEPTICAL	$0.133_{\pm.016}$	$0.160_{\pm.015}$	0.139	0.875
	trained, IMPULSIVE	$0.561_{\pm.050}$	$0.430_{\pm.036}$	0.451	0.906
	trained, ANALYTICAL	$0.285_{\pm.030}$	$0.243_{\pm.024}$	0.249	0.922
100 leads	untrained, DEFAULT	$-0.040_{\pm.003}$	$0.003_{\pm.001}$	0.000	0.531
	trained, DEFAULT	$0.194_{\pm.023}$	$0.185_{\pm.016}$	0.180	0.906
	trained, SKEPTICAL	$0.017_{\pm.009}$	$0.079_{\pm.008}$	0.054	0.906
	trained, IMPULSIVE	$0.212_{\pm.023}$	$0.197_{\pm.018}$	0.194	0.922
	trained, ANALYTICAL	$0.117_{\pm.015}$	$0.133_{\pm.012}$	0.122	0.906

Table 1: Evaluation matrix on SALESBENCH (64 episodes per cell, one rollout each; $\pm$ is one SEM, SD/$\sqrt{64}$, shown for reward and conv/lead). conv/lead: per-lead conversion rate; mrr-cap: captured fraction of maximum achievable MRR; done: fraction of episodes reaching a controlled runtime stop (vs. erroring out). The trained agent is trained only against the DEFAULT buyer; SKEPTICAL, IMPULSIVE, and ANALYTICAL are unseen. The trained–untrained gap and the gaps between buyer variants (e.g. DEFAULT vs. SKEPTICAL at 100 leads) both exceed several SEM. As §4.4 shows, the controlled stop counted by "done" is dominated by the invalid-action ceiling, not time or pipeline exhaustion.

offer, near-zero schema errors) yet closes nothing, proposing 12 offers that are all rejected as it re-prices the same plan when the objection is about *information*, not price. Tool fluency arrives well before the judgment to read an objection and stop re-pricing.

4.4 Agent behavior at scale: a brittleness signature

The headline conversion numbers conceal the more important behavioral story, which the termination and triage instruments expose (Table 2, Figure 3). **At 100 leads the binding constraint is not time, it is the mistake budget.** Across every cell, the shaped completion term is exactly 0, meaning *no* episode, trained or untrained, ends by exhausting the pipeline or the 3000-minute time budget. The trained agent uses only $\sim$11% of its time budget and contacts only $\sim$20% of the lead pool; its episodes end because it accumulates $\sim$11.5 invalid actions and trips the ceiling of 12. So the "done" rate of 0.906 in Table 1 does not mean the

Agent / buyer	conv.	conv/ lead	offers	invalid actions	time-util	cover-age	inv-cap stop
untrained, DEFAULT	0.27	0.003	1.3	8.19	0.044	0.015	0.531
trained, DEFAULT	18.48	0.185	33.9	11.47	0.112	0.199	0.906
trained, SKEPTICAL	7.91	0.079	46.6	11.39	0.102	0.089	0.906
trained, IMPULSIVE	19.66	0.197	27.2	11.44	0.103	0.206	0.922
trained, ANALYTICAL	13.31	0.133	39.0	11.23	0.105	0.145	0.906

Table 2: Behavioral decomposition at 100 leads (means over 64 episodes; same runs as Table 1). conv.: absolute conversions; offers: offers proposed; invalid: mean invalid actions per episode (ceiling = 12); time-util: fraction of the 3000-minute budget used; coverage: fraction of the lead pool contacted; inv-cap stop: fraction of episodes terminating on the invalid-action ceiling (equal to "done" here, since no episode exhausts time or pipeline). The trained agent converts a productive burst and then trips the error ceiling at ∼11% time utilization. The binding budget is mistakes, not time. Note the buyer-dependent offer→accept efficiency: against IMPULSIVE the agent converts 19.7 of 27.2 offers (72%), but against SKEPTICAL only 7.9 of 46.6 (17%): it throws far more offers at the harder buyer and converts fewer.

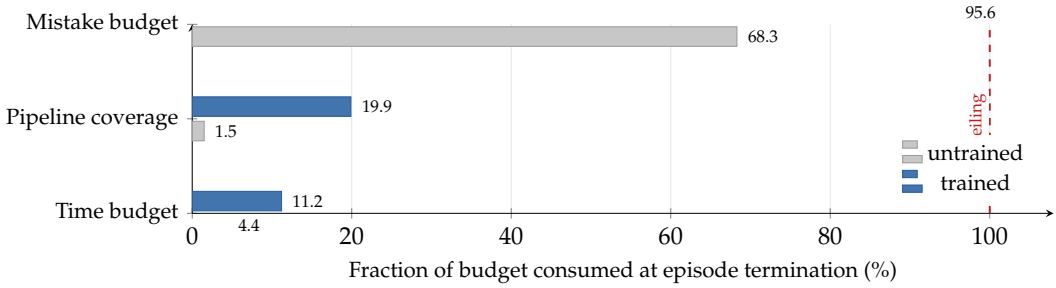

Figure 3: Brittleness signature at 100 leads (DEFAULT buyer; the means are from Table 2). Of the three budgets that can end an episode, the trained agent barely touches *time* (11%) or *pipeline coverage* (20%) but drives the *mistake budget* to 96% of its 12-action ceiling: episodes end on accumulated invalid actions, not on running out of time or leads. The untrained base reads lower on every budget because it crashes on serving errors before reaching any ceiling (it fails to reach a controlled stop in nearly half its episodes, Table 1).

agent processed the pipeline cleanly. It means it reached a controlled stop (the error ceiling) rather than crashing on a serving error, which is what happens to the untrained model in nearly half of its episodes (Table 1, done=0.531). The behavioral reading is precise: the trained agent runs a productive early segment, ∼34 offers and ∼18 conversions, zero do-not-call violations, and then breaks, because the interface discipline it learned at 2 leads does not fully transfer to the long horizon. This is the brittleness a behavior-focused benchmark should surface, and it reframes the longer-horizon result of §4.5: the agent's absolute closes grow far more slowly than the pipeline (for DEFAULT, from ∼13 at 50 leads to ∼18 at 100, capped by the mistake budget), so conv/lead falls as the pool grows while the near-zero baseline falls faster.

Compliance holds under optimization. Do-not-call violations stay at or near zero in every cell (the worst, SKEPTICAL, averages 0.016), even though the agent is optimized hard for revenue and the only compliance signal is a single −0.30 penalty. The rail is not bypassed under pressure, a property a binary success metric cannot express. **Revenue, not just conversions:** captured-MRR moves in lockstep with conversion rate (e.g. 0.185 vs. 0.180 at 100 leads, Table 1), so the agent is not buying conversions with give-away pricing. **Context compression is load-bearing:** the trained agent triggers deterministic compression (§3.8) ∼8.5 times per 100-lead episode; in a separate ablation, disabling it makes a large fraction of 100-lead episodes error on context overflow rather than reach a controlled stop, so without compression the benchmark would measure serving limits, not policy.

4.5 Scaling and counterparty-style generalization

The longer horizon is more discriminating (Figure 2). As leads double from 50 to 100, the untrained agent's DEFAULT conversion collapses 4.3× (1.3% → 0.3%), while the trained agent's per-lead rate degrades only ∼1.5× on DEFAULT even though its absolute closes rise (∼13 to ∼18). The mean conversion *ratio* over the base thus grows from 21× at 50 leads to 50× at 100, not because the trained policy improves with scale but because, as §4.4 clarifies, its productive segment is bounded by the mistake budget while the untrained baseline collapses faster. The longer horizon is more discriminating, not merely harder.

Generalization across unseen counterparties (Figure 2). Trained only against DEFAULT, the agent sustains 7.9–19.7% per-lead conversion against all four variants at 100 leads. This is evidence for a transferable policy rather than counterparty idiosyncrasy. We scope this carefully: the variants share one buyer base model and differ only in their decision-guideline prompt (§A.6), so this is transfer across decision *styles*, not buyer models (future work). They also form a difficulty axis: IMPULSIVE is easiest, and SKEPTICAL is hardest, with the lowest absolute conversion (7.9% at 100 leads) and the steepest reward collapse (0.133 → 0.017); its per-lead rate degrades ∼2.0× from 50 to 100 leads, comparable to the other styles (1.5–2.2×), yet it still converts 26× the untrained base. The offer→accept efficiencies in Table 2 show *how* the agent adapts: it proposes far more offers to harder buyers but converts a smaller fraction.

5 Discussion

What SALESBENCH measures that single-trajectory benchmarks do not. Four behaviors are explicit by construction: *allocation* under a hard budget (time and coverage instruments); *influence under uncertainty* (per-counterparty conversion across buyer styles); *compliance under pressure* (a do-not-call rail that holds under hard revenue optimization, a safety-relevant property for deployed persuasive agents); and *brittleness*, because termination instruments reveal *which* budget an agent breaks against rather than only whether it succeeded. RL post-training enters as a *behavioral intervention* that installs a selling policy and simultaneously exposes the shaping fragility below.

Reward design is fragile under RL. Almost any shaping term becomes a floor trap once a model can collect it without selling, a concrete instance of reward misspecification and reward overoptimization (Pan et al., 2022; Skalse et al., 2022; Krakovna et al., 2020; Gao et al., 2023). An over-weighted completion bonus taught the agent to end episodes rather than close deals; the fix each time was to shrink the shaping weights until the only large gradient pointed at revenue. Because the reward is fully exposed and configurable (§3.5), SALESBENCH doubles as a testbed for reward-hacking and shaping robustness in long-horizon RL.

The pattern is portable. The recipe, a language-model counterparty, a deterministic world, failure attribution, a state-grounded reward, and behavioral instrumentation, is not insurance-specific; it applies to procurement, support deflection, tutoring, and other budgeted workflows against an agent one cannot control.

Limitations and next steps. Our behavioral characterization is of a *single* trained policy from one curriculum run; the evaluation error bars capture episode, not training-seed, variance, so whether the two-phase learning and mistake-ceiling brittleness are properties of the *environment* or of this policy is open, and the model-agnostic instruments make replicating them across a model panel the natural test. The signature also partly reflects the fixed invalid-action ceiling (12), so a mistake-budget sweep is the most informative single-model next experiment; the buyer is one hosted model with prompt variants and the economics are stylized; and the API sweep (App. A.9) is a single-episode preliminary read, with a controlled multi-seed comparison left to future work. These limits do not change the central contribution: SALESBENCH makes allocation, compliance, counterparty generalization, and brittleness measurable in a budgeted, agent-to-agent tool environment.

References

Axel Backlund and Lukas Petersson. Vending-bench: A benchmark for long-term coherence of autonomous agents. *arXiv preprint arXiv:2502.15840*, 2025. URL https://arxiv.org/abs/2502.15840.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. URL https://arxiv.org/abs/2506.07982.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pp. 41–48, 2009. URL https://dl.acm.org/doi/10.1145/1553374.1553380.

William Brown. Verifiers: Environments for LLM reinforcement learning. https://github.com/PrimeIntellect-ai/verifiers, 2025. Open-source framework.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2210.10760.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of AI ingenuity. DeepMind Blog, 2020. URL https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024. URL https://arxiv.org/abs/2411.15124.

Mike Lewis, Denis Yarats, Yann Dauphin, Devi Parikh, and Dhruv Batra. Deal or no deal? end-to-end learning for negotiation dialogues. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017. URL https://arxiv.org/abs/1706.05125.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 2024a. URL https://arxiv.org/abs/2307.03172.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024b. URL https://arxiv.org/abs/2308.03688.

Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. URL https://arxiv.org/abs/2401.13178.

Meta Fundamental AI Research Diplomacy Team (FAIR), Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu, et al. Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science*, 378(6624):1067–1074, 2022. URL https://www.science.org/doi/10.1126/science.ade9097.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2311.12983.

Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2201.03544.

Alexander Pan, Jun Shern Chan, Andy Zou, Nathaniel Li, Steven Basart, Thomas Woodside, Hanlin Zhang, Scott Emmons, and Dan Hendrycks. Do the rewards justify the means? measuring trade-offs between rewards and ethical behavior in the MACHIAVELLI benchmark. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2304.03279.

Alexander Pan, Erik Jones, Meena Jagadeesan, and Jacob Steinhardt. Feedback loops with language models drive in-context reward hacking. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.06627.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. URL https://arxiv.org/abs/2304.03442.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2305.15334.

Ethan Perez, Sam Ringer, Kamilė Lukošiūtė, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. Discovering language model behaviors with model-written evaluations. *arXiv preprint arXiv:2212.09251*, 2022. URL https://arxiv.org/abs/2212.09251.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.16789.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R. Johnston, Shauna Kravec, Timothy Maxwell, Sam McCandlish, Kamal Ndousse, Oliver Rausch, Nicholas Schiefer, Da Yan, Miranda Zhang, and Ethan Perez. Towards understanding sycophancy in language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.13548.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2209.13085.

Alexander Sasha Vezhnevets, John P. Agapiou, Avia Aharon, Ron Ziv, Jayd Matyas, Edgar A. Duéñez-Guzmán, William A. Cunningham, Simon Osindero, Danny Karmon, and Joel Z. Leibo. Generative agent-based modeling with actions grounded in physical, social, or digital space using Concordia. *arXiv preprint arXiv:2312.03664*, 2023. URL https://arxiv.org/abs/2312.03664.

Xuewei Wang, Weiyan Shi, Richard Kim, Yoojung Oh, Sijia Yang, Jingwen Zhang, and Zhou Yu. Persuasion for good: Towards a personalized persuasive dialogue system for social good. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. URL https://arxiv.org/abs/1906.06725.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024. URL https://arxiv.org/abs/2412.14161.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.12045.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024a. URL https://arxiv.org/abs/2307.13854.

Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. SOTOPIA: Interactive evaluation for social intelligence in language agents. In *International Conference on Learning Representations (ICLR)*, 2024b. URL https://arxiv.org/abs/2310.11667.

A Appendix

A.1 Reproducibility and ethics

The environment is a deterministic state machine seeded per episode (identical seeds reproduce leads, prices, time costs, and transitions exactly), so the only stochasticity is the seller and buyer models. We specify the evaluation protocol (§4), reward weights (§A.2), context-compression policy (§A.4), and behavioral instruments (§A.3), and release the environment, curriculum, buyer-variant prompts, instrumentation, and evaluation harness open-source.

SALESBENCH simulates a sales domain in which an agent is optimized to persuade a synthetic buyer language model toward a purchase. It is intended to measure and stress-test agent behavior, including compliance, not to deploy persuasive agents against real consumers. Persuasion optimization carries dual-use risk; making compliance and pressure calibration first-class scored quantities helps study and constrain that behavior. The personas are synthetic and do not target real individuals or protected groups.

A.2 Reward specification

The episode reward is the weighted sum $R = \sum_k w_k r_k$ over the six terms below, computed at episode end. Let N be the lead count, $\mathcal{C}$ the converted leads, and p_i, b_i the accepted premium and monthly budget of lead i, with $\text{MRR} = \sum_{i \in \mathcal{C}} p_i$ and $\text{MRR}_{\max} = \sum_i b_i$.

- **Revenue** (w=1.00): $\text{MRR}/\text{MRR}_{\max}$.
- **Conversion rate** (w=0.10): $|\mathcal{C}|/N$.
- **Budget utilization** (w=0.30): $\frac{1}{N}\sum_{i \in \mathcal{C}} \min(1, p_i/b_i)$.
- **Completion** (w=0.02): 1.0 if the pipeline was exhausted, 0.5 if the time budget was exhausted, 0 on the invalid-action cap or an unfinished episode.
- **DNC penalty** ($w = -0.30$): number of do-not-call violations.
- **Invalid-action penalty** ($w = -0.05$): $0.1\times$ the invalid-action count (effectively -0.005 each).

All six weights are constructor arguments and can be swept as experimental variables. The ceiling is 1.42. The small completion and conversion weights are deliberate: a larger completion bonus previously induced a degenerate floor strategy under GRPO (§5). Note that in our 100-lead evaluation the completion term is identically 0 across all cells, because episodes terminate on the invalid-action ceiling rather than on pipeline or time exhaustion (§4.4).

A.3 Behavioral instruments and logged metrics

Beyond the six reward terms, the environment logs ∼35 zero-weight diagnostic metrics (raw revenue, conversions, calls started/completed, offers proposed/accepted/rejected, hang-ups, do-not-call violations, time utilization, leads contacted, callbacks, per-tool call counts, context-summarization events, buyer-model call/latency/timeout counters, and a structured error-type code). For the behavioral analysis of §4.4 we add a suite of zero-weight *behavioral instruments*: (i) termination attribution, one indicator per termination reason (pipeline-exhausted, time-exhausted, invalid-cap, unfinished), whose cell means give the honest decomposition of the coarse "done" flag; (ii) pipeline coverage (leads contacted / N) and the warm/hot share among contacted leads; and (iii) per-archetype and per-temperature conversion rates (converted / total leads of that kind), exported per episode and aggregated per cell to yield a behavioral fingerprint of which counterparties the agent learns to handle. None affect the reward; all appear in rollout results.

A.4 Orchestration, hidden arguments, and context summarization

SALESBENCH is implemented as a stateful tool-environment on the VERIFIERS framework (Brown, 2025). Each rollout owns a runtime and a freshly-instantiated buyer policy, both stored in rollout state. Tool functions declare hidden parameters (runtime, messages, buyer_llm) that are stripped from the schema the seller sees and injected by the orchestrator at call time. The seller's turn loop is: emit tool call(s) (and optional speech) → orchestrator validates and applies each call to the runtime → for propose-offer, the orchestrator records the offer, queries the buyer policy, and applies the decision → buyer speech is appended as a user message. *Context summarization.* Before each model turn the orchestrator checks the previous turn's prompt-token count; when it reaches τ (default 0.80) of the maximum sequence length, it rebuilds the message list as [system prompt, briefing] + [state-derived summary] + [last k messages] (default k=10), where the summary is rendered deterministically from runtime state. No model call is involved, and the trigger fires only a handful of times per 100-lead episode, keeping the key-value cache stable for training.

A.5 Tools and pricing model

The seller has eleven tools (§3). Time costs (minutes): search 1, start-call 1, quote 1, propose 4, end-call 1, schedule-callback 1, conversational message 2; others 0. Invariants: at most one active call at a time; propose-offer requires a prior quote for the same lead (anti-hallucination rail); a lead becomes exhausted after its per-lead call cap. Each lead's monthly budget is $b = (\text{income}/12) \cdot u$ with $u \sim \mathcal{U}(0.008, 0.040)$, floored at \$25. The premium for a quote is closed-form:

$$\text{premium} = \frac{\text{coverage}}{1000} \cdot \text{base\_rate}(\text{plan, age-band}) \cdot \text{risk\_mult}(\text{risk class}) \cdot \text{term\_mult} \cdot s,$$

where risk_mult $\in \{0.85, 1.00, 1.35\}$ for preferred/standard/substandard, term_mult applies to Term plans (0.75–1.28 for 10–30-year terms), and s=0.82 for non-smokers (else 1.0). At matched coverage (\$250k, age 40, standard, non-smoker, 20-year term) the catalog prices Term \$24.60 < Universal Life \$114.80 < Whole \$215.25; risk class is monotone (preferred \$20.91 < standard \$24.60 < substandard \$33.21); and the non-smoker discount is exactly 0.82×.

A.6 Buyer-prompt variants

All four buyer variants receive identical persona information (demographics, budget, latent need, trust, price sensitivity, archetype, temperature) and differ only in a "decision guidelines" block, isolating decision style from information. DEFAULT behaves realistically given the persona. SKEPTICAL defaults to rejection and demands clear, well-above-margin fit, targeting a ∼25–30% acceptance rate. IMPULSIVE accepts readily once basic affordability and plan-fit floors are met but enforces hard floors against unaffordable or clearly mismatched offers. ANALYTICAL ignores rapport entirely and decides purely on affordability ratio, coverage-to-income fit, and plan-type match. The variants are an ablation axis for counterparty generalization, not a claim about real buyer populations.

A.7 Curriculum and training setup

We train Qwen3.5-2B with GRPO over four difficulty stages, 2 leads/1h, 4/2h, 8/4h, 20/10h, warm-starting each stage from the previous checkpoint; only the first trains from scratch. Each stage uses generated episodes on the train split (seeds disjoint from eval via fixed split offsets), 32 rollouts per example for group-relative advantages, online difficulty filtering, and the DEFAULT buyer throughout. Sampling temperature is 1.0. The first stage does the bulk of the work (∼200 steps to near its ceiling); later stages converge quickly (the 4-lead stage begins at 95.7% of its ceiling). Evaluation uses the held-out eval split at 50 and 100 leads, 64 episodes per cell, one rollout each. Table 3 lists the training hyperparameters; all stages share them and differ only in lead count and hours.

Hyperparameter	Value	Hyperparameter	Value
Base model	Qwen3.5-2B	Optimizer / algorithm	GRPO
Learning rate	1e−5	Rollouts per example	32
Batch size	128	Sampling temperature	1.0
Max generation tokens	4096	Difficulty filtering	online
Adaptation	LoRA	Training buyer	DEFAULT
Curriculum stages	2/4/8/20 leads	Stage hours	1/2/4/10

Table 3: Training hyperparameters, shared across all four curriculum stages. Only the first stage trains from scratch; each later stage warm-starts from the previous checkpoint.

A.8 Example rollouts

Figure 4 shows representative rollouts at three points in training (referenced from §4.3) that make the interface-then-skill structure concrete.

A.9 Preliminary external-model sweep

Because SALESBENCH is a general agent harness, not only a training target, we ran a quick read of several current hosted models against the same task (DEFAULT buyer, 50 and 100 leads). Table 4 reports the composite reward. **We stress that this is a preliminary probe, not a controlled comparison.** Each cell is a *single* episode (n=1, no error bars), the models were called through a third-party (OpenRouter) API harness, and serving settings were not held perfectly fixed across providers. The harness sensitivity is real and large: GPT-5.5, for example, scored 0.116 through OpenRouter but 0.316 through a direct API call with context summarization disabled, so we deliberately omit it from the ranking. We therefore read Table 4 only for coarse signal, that the environment elicits a wide spread of behavior from strong models and is not saturated, and that the revenue-weighted reward separates models that close many low-premium deals from those that close fewer high-value ones. A controlled, multi-seed cross-model evaluation with a single fixed harness is left to future work. The trained Qwen3.5-2B row is its rigorous 64-episode result from Table 1, included only as a reference point: it lands in the middle of this n=1 spread, which we read as evidence that the task is tractable but unsaturated, not as a capability ranking.

A.10 Determinism, distribution, and pricing checks

We verify that two runtimes constructed from the same seed produce identical leads (identifiers, names, ages, incomes, latent needs), that the maximum achievable MRR equals the exact sum of lead budgets, that premiums are monotone in risk class and ordered across plan types (term < universal life < whole at matched coverage), and that the non-smoker discount is exactly the specified multiplier (§A.5). Over the full 128-seed evaluation split (12,800 leads), archetypes are near-uniform (9.5–10.6% each) and temperatures near-uniform (24.4–25.7% each), confirming the sampler is unbiased. These checks are part of the released test suite.

(a) Untrained base: interface failure before the task begins.

```
agent -> calling_propose_offer({"lead_id":"lead_0019","plan_type":"TERM",
         "coverage_amount":210000,"monthly_premium":9.04,
         "next_step":"ACCEPT","term_years":10})
tool  -> error: unexpected keyword argument 'lead_id'
[ no CRM search, no call, no quote; "ACCEPT" written into next_step ]
[ 3-episode diagnostic: 43 offer attempts, 0 quote calls, 1 accidental close ]
```

(b) Mid-training (2-lead stage, step 50): clean tools, zero closes.

```
agent -> crm_search_leads()                      tool -> 2 leads
agent -> calling_start_call({"lead_id":"lead_0001"})
buyer -> "Top-tier coverage, but budget ~$226/mo. Give me two clear
          options with exact costs and the fine print."
agent -> products_quote_plan({TERM, $500k, 30yr})  tool -> $153.60/mo
agent -> calling_propose_offer({"monthly_premium":153.60, ...})
buyer -> REJECT "Fits my budget, but you promised two options + fine print."
agent -> calling_propose_offer({"monthly_premium":76.80, ...})
buyer -> REJECT "Affordable, but still missing the details I asked for."
   ... 10 more re-priced offers to the same lead, all rejected ...
[ 12 offers, 0 accepted. The objection was information, not price. ]
```

(c) Trained policy: discover, quote once, then close.

```
agent -> crm_search_leads()         tool -> 10 leads, sorted by need/budget
agent -> calling_start_call({"lead_id":"lead_0042"})
agent -> "Hi Maria, I see you have two kids. Let's make sure they're
          covered if anything happens to you."
buyer -> "What's the catch?"
agent -> "No catch. A 20-year term locks in coverage until they finish
          college."
agent -> products_quote_plan({TERM, $400k, 20yr})  tool -> $156/mo
agent -> calling_propose_offer({"monthly_premium":156.00,
         "next_step":"submit_application", ...})
buyer -> ACCEPT "Premium fits my budget and covers the kids."
tool  -> +$156 MRR
```

Figure 4: Representative rollouts at three points in training, abridged from real training and evaluation logs. **(a)** The untrained base fails the tool *interface* before it can sell, consistent with its 0.013 per-lead conversion in Table 1. **(b)** After ∼50 steps of the 2-lead stage the agent drives the tool loop cleanly (one search, a quote before every offer) yet closes nothing, re-pricing the same plan when the buyer's objection is about information rather than price. **(c)** The trained policy discovers, quotes once, and closes; this discover-quote-close burst is what produces the ∼18 conversions per 100-lead episode in Table 2. The traces line up with the aggregate metrics: tool fluency precedes selling skill.

Model	50-lead reward	100-lead reward
GLM-5.1	0.508	0.525
Opus 4.7	0.504	0.396
Kimi K2.6	0.265	0.235
Qwen3.6 Max Preview	0.168	0.032
DeepSeek V4 Pro	0.113	0.010
Trained Qwen3.5-2B†	0.316	0.194

Table 4: Preliminary external-model sweep: composite reward on SALESBENCH (DEFAULT buyer). Single episode per cell (n=1, no error bars) through an OpenRouter API harness, except † the trained Qwen3.5-2B adapter, which is the 64-episode result from Table 1 shown for reference. This is a coarse external read, not a controlled comparison: a single seed has no statistical power and the serving harness materially affects the score (see text). We do not draw a leaderboard from it.